Generative AI for Mechanical Engineering, Part I

Lightweight TensorFlow Workflows for Unsupervised Learning, Generative Models, Natural Language Processing, Attention, Transformers, and Simple Multi-Modal Models

A beginner-friendly tutorial for Mechanical Engineering students

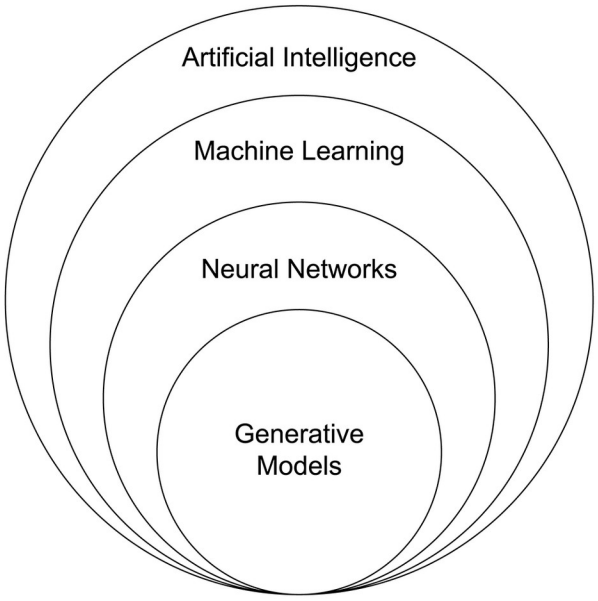

Figure 1. Generative models sit inside the broader family of artificial intelligence and machine learning ideas. This visual sets the scope for the tutorial: small generative workflows rather than large production systems. Source: The Original Benny C (2023), Wikimedia Commons, CC BY-SA 4.0. Citation: The Original Benny C, 2023

Prepared for classroom use in current Google Colab-style environments

Date: April 2026

Detailed Teaching Outline

The tutorial begins with intuition, then moves to small TensorFlow examples, then connects each idea to mechanical engineering data. The sequence below is designed for students who can read simple Python but have not studied generative artificial intelligence, unsupervised learning, natural language processing, attention, transformers, or multi-modal learning before.

Table 1. Teaching sequence used in this tutorial.

Sequence	Main idea	Classroom activity	Mechanical Engineering connection
1	Why generative AI matters	Compare supervised, unsupervised, and generative workflows.	Recognize that engineering images, notes, and sensor records are often unlabeled.
2	Python and TensorFlow mounting bases	Review arrays, tensors, strings, functions, classes, plotting, and model inputs and outputs.	Prepare to handle inspection images, maintenance notes, and tabular metadata.
3	Unsupervised learning and representations	Study clustering, compression, and latent spaces using visuals and small code.	Discover structure in defect images without manually assigning labels.
4	Autoencoders	Train a small encoder-bottleneck-decoder network and inspect reconstructions.	Compress and reconstruct simple crack-like image arrays.
5	Generative adversarial networks	Build a tiny generator and discriminator and monitor generated samples.	Create proof-of-concept synthetic defect-like images for learning and augmentation discussion.
6	Diffusion-style denoising	Add noise, train a denoiser, and repeat denoising steps.	Connect to image enhancement, noisy sensor cleanup, and future restoration systems.
7	Beginner natural language processing	Tokenize short notes, build a vocabulary, learn embeddings, classify notes, and predict next tokens.	Process inspection comments, repair logs, and maintenance descriptions.
8	Attention and transformers	Visualize attention weights and build a small transformer encoder.	Understand how a model can focus on important words in inspection sentences.
9	Simple multi-modal models	Combine image and text branches in one small TensorFlow model.	Link inspection images with textual descriptions and identify matched versus mismatched pairs.
10	Synthesis and responsible use	Compare model families and propose safe beginner projects.	Prepare for future mechanical engineering generative AI workflows while respecting validation needs.

Table of Contents

This static table of contents lists the learning path. Page numbers may differ after local editing or printing.

1. Introduction to Generative AI in Mechanical Engineering

Generative artificial intelligence, or generative AI, is a family of methods that learn patterns from data and then produce new outputs that resemble, complete, reconstruct, transform, or describe data. A generative model can create a new image-like sample, reconstruct a noisy signal, continue a text sequence, or connect an image with a sentence. In this tutorial, the emphasis is not on large commercial systems. The emphasis is on small, inspectable TensorFlow workflows that help students understand the first principles behind modern generative AI.

Mechanical Engineering produces many forms of data: photographs of cracks, spalls, corrosion, deflection, and machined-surface defect; handwritten or typed inspection comments; mechanical component and equipment maintenance logs; laboratory measurements; sensor records; drawings; and geospatial information. Much of this data is not neatly labeled. An inspector may take thousands of photographs before anyone assigns detailed defect categories. A maintenance database may contain short notes with inconsistent wording. Unsupervised and generative workflows help students see how patterns can be learned from such data even when every example does not have a human label.

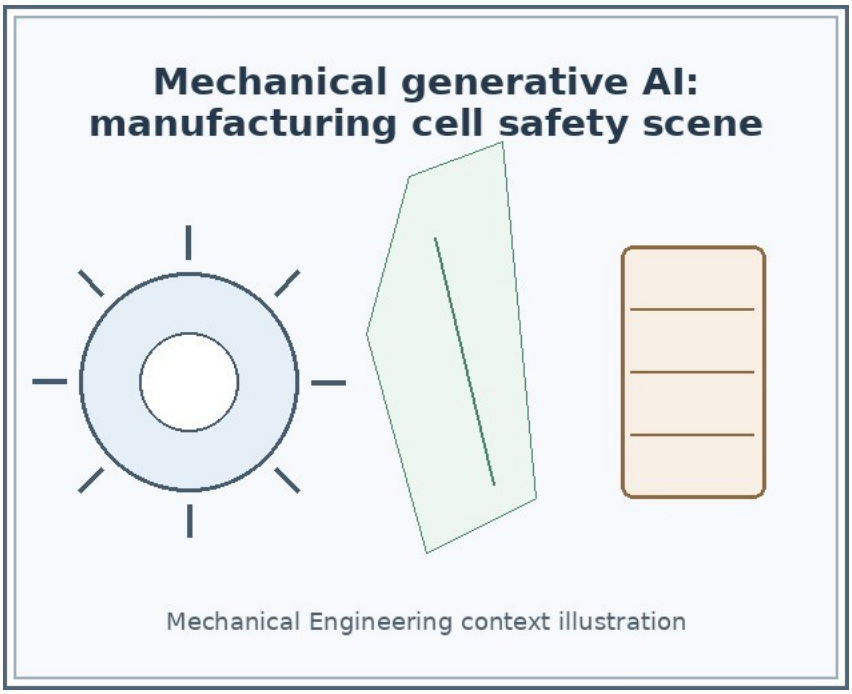

Figure 2. A complete mechanical engineering inspection photograph showing cracks on the Miami robotic arm bracket. Real inspection imagery motivates image compression, reconstruction, denoising, and image-text pairing workflows. Source: National Vehicle systems Safety Board (2018), Wikimedia Commons, public domain. Citation: NTSB, 2018

1.1 What generative AI does

At a beginner level, a generative workflow can be understood as a model that learns a useful internal pattern and then uses that pattern to create or reconstruct something. The output is not copied from one training example. Instead, the model uses learned structure. For example, an autoencoder learns a compressed representation of an image and reconstructs the input. A generative adversarial network, or GAN, learns to create new image-like samples from random numbers. A diffusion-style denoising workflow learns to remove noise step by step. A text model learns word patterns and predicts a class or a next token. A multi-modal model learns from more than one data type, such as a machined surface image and a short text description.

Table 2. Supervised, unsupervised, and generative workflows compared for beginners.

Workflow	Training data	Main question	Typical output	Mechanical Engineering example
Supervised learning	Inputs with labels	Can the model predict the known label?	Class, number, or category	Classify inspection images as crack, spall, corrosion, or no visible defect.
Unsupervised learning	Inputs without labels	Can the model discover useful structure?	Groups, compressed representations, reconstructions, or patterns	Cluster machined surface images by visual similarity before labeling them.
Generative learning	Examples of a data type, sometimes with text or labels	Can the model produce, reconstruct, complete, or transform data?	Generated sample, reconstruction, denoised sample, or text continuation	Generate proof-of-concept defect-like images or denoise a noisy inspection image.

1.2 Why unlabeled data matter

A label is an extra piece of information that tells the model what an example means. In a supervised image classifier, a label might be "crack" or "no crack." Labels are valuable but expensive. They require time, consistency, expert judgment, and quality control. Many mechanical engineering data archives contain more raw data than labels. Unsupervised learning is useful because it can learn representations from raw data before a detailed labeling process exists.

A representation is a compact description that a model uses internally. For a crack image, a representation might encode direction, thickness, roughness, brightness, and background texture. For a maintenance note, a representation might encode words such as "leak," "joint," "bearing," "spall," and "urgent." The representation is often more useful than the original raw data when we want to compare examples, detect unusual cases, or build a later supervised system.

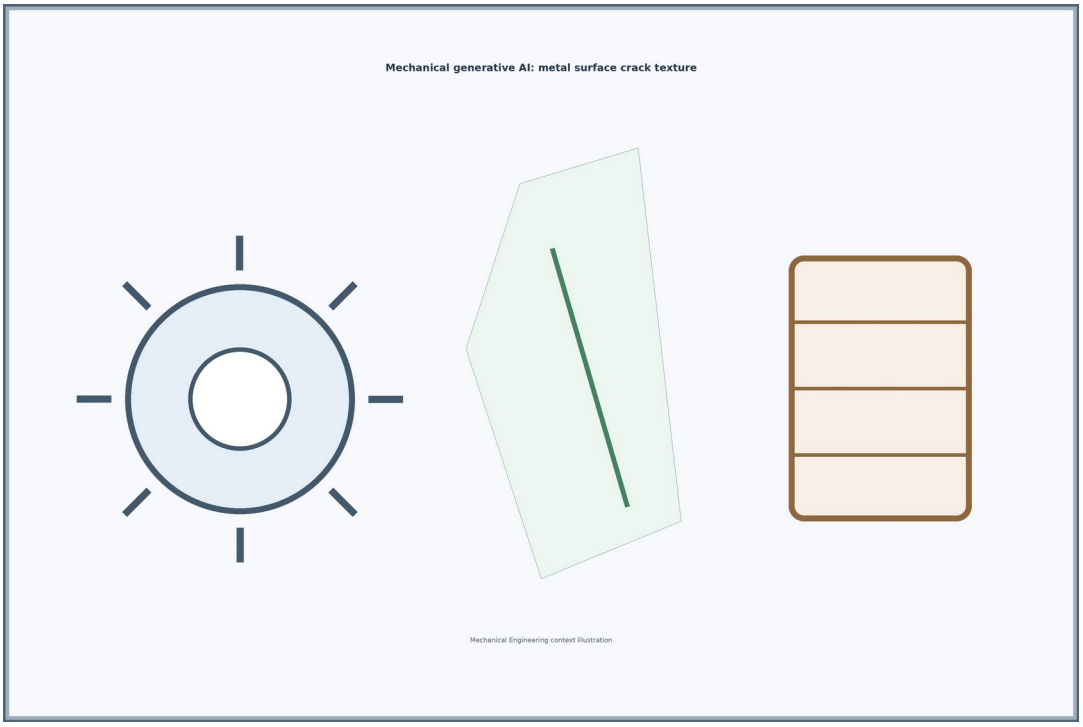

Figure 3. A complete photograph of cracked coated steel. This type of visual data can motivate small classroom examples for image compression, reconstruction, denoising, and text-image pairing. Source: Marek Slusarczyk (2012), Wikimedia Commons, CC BY 3.0. Citation: Slusarczyk, 2012

1.3 Why lightweight TensorFlow examples are useful

TensorFlow and Keras provide high-level tools for building neural networks with relatively compact code. Official TensorFlow tutorials include examples for autoencoders, deep convolutional GANs, text classification, transformers, and image captioning workflows (TensorFlow, 2024a; TensorFlow, 2024b; TensorFlow, 2024c; TensorFlow, 2024d; TensorFlow, 2024e). Keras examples are designed as focused demonstrations and many are usable in Colab-style environments (Keras, 2024a).

Lightweight examples are important for beginning students because they make the learning goal visible. A small model can be trained, inspected, and modified during a class session. The goal is not to create a certified mechanical mechanical systems tool. The goal is to understand inputs, outputs, losses, representations, and visual checks. This basis prepares students to evaluate future systems more critically.

Safety and professional context

The examples in this tutorial are educational prototypes. They are not validated engineering decision tools. Mechanical mechanical systems decisions require domain expertise, documented data quality, uncertainty analysis, field verification, and professional responsibility.

1.4 Major learning outcomes

- Explain the basic ideas of unsupervised learning and introductory generative AI using lightweight TensorFlow workflows.
- Implement beginner-level TensorFlow examples of autoencoders, GANs, diffusion-style denoising, text processing, attention, transformers, and simple multi-modal models.
- Interpret small image, text, and image-text model outputs in mechanical engineering contexts.
- Use visual inspection, reconstruction error, loss curves, and simple predictions to understand model behavior.
- Distinguish classroom proof-of-concept workflows from validated mechanical engineering applications.

Section summary

Generative AI is useful for mechanical engineering education because many engineering data sources are visual, textual, noisy, and incompletely labeled. Small TensorFlow examples help students learn the ideas without relying on large pretrained systems or heavy computation.

2. Python and AI Workflow Mounting bases

This section reviews only the Python and machine learning vocabulary needed to follow the examples. The code is written for a notebook-style environment such as Google Colab. Each code cell can be copied into a notebook and run in order. The examples use small arrays, small neural networks, and visual inspection. A graphics processing unit, or GPU, can make training faster, but the examples are intentionally small.

Table 3. Beginner Python and AI terms used throughout the tutorial.

Term	Plain-language meaning	Example in this tutorial
Variable	A named container for a value.	image_size = 28 stores the image width

		and height.
Array	A rectangular collection of numbers.	A grayscale image can be stored as a 28 by 28 array.
Tensor	A TensorFlow array that can flow through a neural network.	A batch of images has shape (batch, height, width, channels).
String	Text data enclosed in quotation marks.	"crack observed near joint".
Table	Rows and columns of structured information.	A maintenance log with asset ID, date, note, and action.
Function	Reusable code that takes inputs and returns outputs.	make_crack_images(n) returns n toy image arrays.
Class	A reusable blueprint for an object that has data and behavior.	A custom TransformerEncoder layer defines repeated neural-network operations.
Method	A function that belongs to an object.	model.fit(...) trains a Keras model.
Argument	A value passed into a function or method.	epochs=5 tells model.fit how many passes to train.
Plot	A visual display of data or model output.	imshow displays an image array.

2.1 Inputs, outputs, and intermediate results

Table 4. Data objects that appear in generative AI workflows.

Object	Meaning	Mechanical Engineering example
Input data	The raw example given to a model.	A 28 by 28 grayscale crack-like image or an inspection sentence.
Output data	The result produced by the model.	A reconstructed image, a predicted class, or a similarity score.
Latent representation	A compressed internal vector learned from an input.	A 16-number vector summarizing a crack-like image.
Embedding	A learned vector representation of a token, image, or other item.	A word such as "joint" becomes a vector that can be compared to other words.
Prompt	Text used to guide or ask a model for a response.	A phrase such as "inspection note about cracking". In this tutorial, prompts are explained conceptually, not used for large systems.
Prediction	A model estimate.	Probability that a note describes an urgent maintenance issue.
Reconstruction	A model output that tries to reproduce the input.	Autoencoder output compared with the original image.
Generated sample	A new sample created by a generative model.	A GAN output that looks like a simple crack-like pattern.

2.2 Minimal setup for TensorFlow examples

The following imports are used repeatedly. The code sets a random seed so that classroom outputs are more reproducible. Small differences can still occur because neural-network training includes randomness.

Code Example 1. Notebook setup

```
# Lightweight setup for Colab-style notebooks
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
```

```
from tensorflow.keras import layers

# Make results more repeatable for classroom demonstrations.
tf.keras.utils.set_random_seed(42)

print("TensorFlow version:", tf.__version__)
```

2.3 Arrays, tensors, and images

A grayscale image can be stored as a two-dimensional array. A neural network usually expects a batch of images with a channel dimension, so a batch of grayscale images has shape (number_of_images, height, width, 1).

Code Example 2. From array to tensor

```
# A tiny 4 by 4 grayscale image represented as numbers.
small_image = np.array([
    [0.0, 0.0, 0.2, 0.0],
    [0.0, 0.5, 1.0, 0.3],
    [0.1, 1.0, 0.6, 0.0],
    [0.0, 0.2, 0.0, 0.0]
], dtype="float32")

# TensorFlow can convert the NumPy array into a tensor.
small_tensor = tf.convert_to_tensor(small_image)
print("Array shape:", small_image.shape)
print("Tensor shape:", small_tensor.shape)

plt.imshow(small_image, cmap="gray")
plt.title("Toy grayscale image")
plt.axis("off")
plt.show()
```

2.4 A small mechanical engineering toy image dataset

To keep the tutorial independent of large downloads, several image examples use synthetic toy arrays that resemble very simple crack-like patterns. These arrays are not real inspection data. They are small classroom objects for learning model mechanics.

Code Example 3. A tiny image generator for classroom arrays

```
def make_crack_images(n_images=512, image_size=28, noise=0.08):
    """Create small grayscale toy images with simple crack-like lines."""
    images = np.zeros((n_images, image_size, image_size, 1), dtype="float32")
    for i in range(n_images):
        x = np.random.randint(4, image_size - 4)
        y = np.random.randint(4, image_size - 4)
        length = np.random.randint(8, 18)
        slope = np.random.choice([-1, 0, 1])
        for step in range(length):
            xx = np.clip(x + step, 0, image_size - 1)
            yy = np.clip(y + slope * step + np.random.randint(-1, 2), 0, image_size - 1)
            images[i, yy, xx, 0] = 1.0
            if yy + 1 < image_size:
                images[i, yy + 1, xx, 0] = 0.6
        images[i] += noise * np.random.randn(image_size, image_size, 1)
    return np.clip(images, 0.0, 1.0)

x_images = make_crack_images(16)
fig, axes = plt.subplots(2, 8, figsize=(8, 2))
for ax, img in zip(axes.ravel(), x_images):
```

```
    ax.imshow(img.squeeze(), cmap="gray")
    ax.axis("off")
plt.suptitle("Toy crack-like image arrays")
plt.show()
```

Input meaning: n_images controls how many examples are created. Output meaning: the function returns a tensor-shaped NumPy array with values between 0 and 1. Mechanical engineering interpretation: the images are deliberately simplified, but they create an easy place to learn compression, reconstruction, generation, and denoising before using real inspection data.

Section summary

The core data types are arrays, tensors, strings, and learned vectors. The examples will reuse simple image arrays, text strings, and Keras models. The same logic later extends to real engineering data after careful data management and validation.

3. Part I: Unsupervised Learning and Introductory Generative Models

Part I focuses on learning from data without requiring a label for every example. The main idea is that unlabeled examples can still reveal structure. An autoencoder learns to compress and reconstruct inputs. A GAN learns by a contest between a generator and a discriminator. A diffusion-style denoising workflow learns to reverse noise. These workflows teach representation learning, reconstruction, sampling, and visual inspection.

Table 5. Part I model families and their beginner purpose.

Family	Learns from	Beginner output	What students inspect	Mechanical Engineering relevance
Autoencoder	Input examples, often unlabeled	Reconstruction and latent vector	Original versus reconstructed images and reconstruction loss	Image compression, anomaly screening, denoising mounting bases.
GAN	Real examples and random noise vectors	Generated samples	Sample quality, generator loss, discriminator loss	Synthetic defect-like samples for proof-of-concept augmentation discussion.
Diffusion-style denoising	Clean examples with added noise	Denoised samples and step snapshots	Noise level, denoising quality, repeated refinement	Image enhancement, sensor denoising, restoration intuition.

4. Introduction to Unsupervised Learning

Unsupervised learning uses examples without target labels. Instead of asking the model to predict a known answer, we ask it to organize, compress, reconstruct, or represent data. The model may discover clusters, directions of variation, compressed features, or unusual examples. This is valuable when labeling is costly, uncertain, or not yet available.

In mechanical engineering, unlabeled data may include thousands of machined surface photographs, turbine blade surface images, or short inspection notes. Before building a supervised classifier, an engineer may want to know whether the images contain visually distinct groups. Do some images have long diagonal cracks? Do some have noisy lighting? Do some notes mention bearings, joints, vibration, wear, or deflection? Unsupervised learning can help organize such questions.

4.1 Unlabeled data and representation learning

A representation is a useful summary of an example. If the raw input is a large image, the representation might be a much shorter vector. If the raw input is a sentence, the representation might be a vector that summarizes its meaning. Representation learning means that the model learns these summaries from data rather than requiring humans to hand-design every measurement.

Table 6. What unlabeled data can still teach a model.

Goal	Plain-language question	Typical method	Mechanical Engineering interpretation
Discover groups	Which examples look similar?	Clustering or embedding visualization	Group similar crack patterns before detailed labeling.

Compress data	Can a smaller vector keep the important information?	Autoencoder bottleneck	Store a compact representation of an inspection image.
Reconstruct data	Can the model rebuild the input?	Autoencoder decoder	Identify whether a reconstruction misses unusual defect features.
Generate samples	Can the model create plausible examples?	GAN or diffusion-style workflow	Explore synthetic images for education or data augmentation discussion.
Remove noise	Can the model recover a cleaner signal?	Denoising autoencoder or diffusion-style denoiser	Improve a noisy laboratory image or sensor-like signal for visualization.

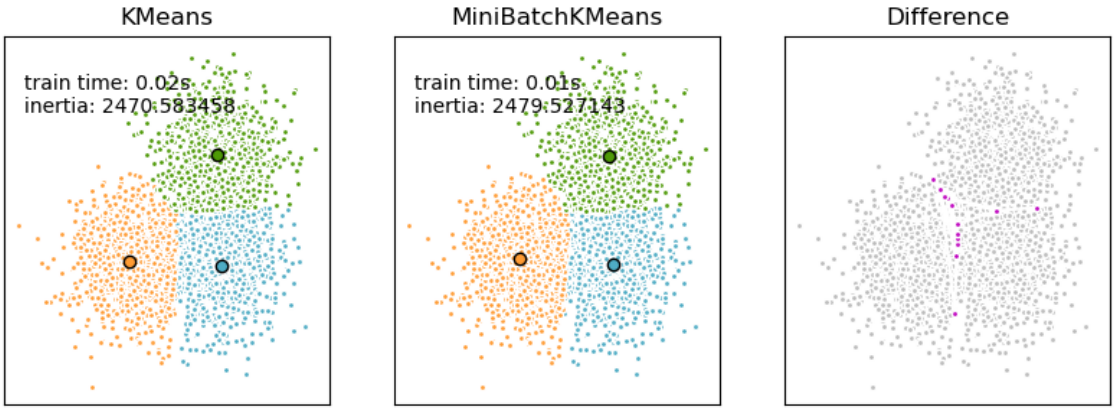

Figure 4. K-means and mini-batch K-means are classic unsupervised clustering methods. The figure shows the kind of visual grouping that helps students understand structure discovery before neural generative models. Source: scikit-learn developers (2024), scikit-learn documentation. Citation: scikit-learn developers, 2024

4.2 Visualizing structure

Humans often understand unsupervised learning better through plots. A high-dimensional input can be mapped to two dimensions for visualization. The plot does not prove that the data are perfectly organized, but it can reveal patterns worth investigating. For mechanical engineering imagery, a plot may show that certain photographs group by lighting, texture, crack direction, or camera angle. This is useful because it warns us that a model may learn photography conditions as well as engineering content.

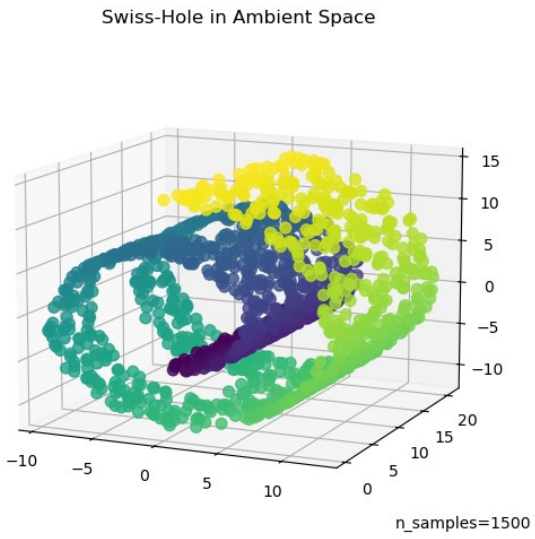

Figure 5. A three-dimensional Swiss roll dataset used in machine learning education. The example shows data lying on a curved surface in a higher-dimensional space. Source: scikit-learn developers (2025), scikit-learn documentation. Citation: scikit-learn developers, 2025

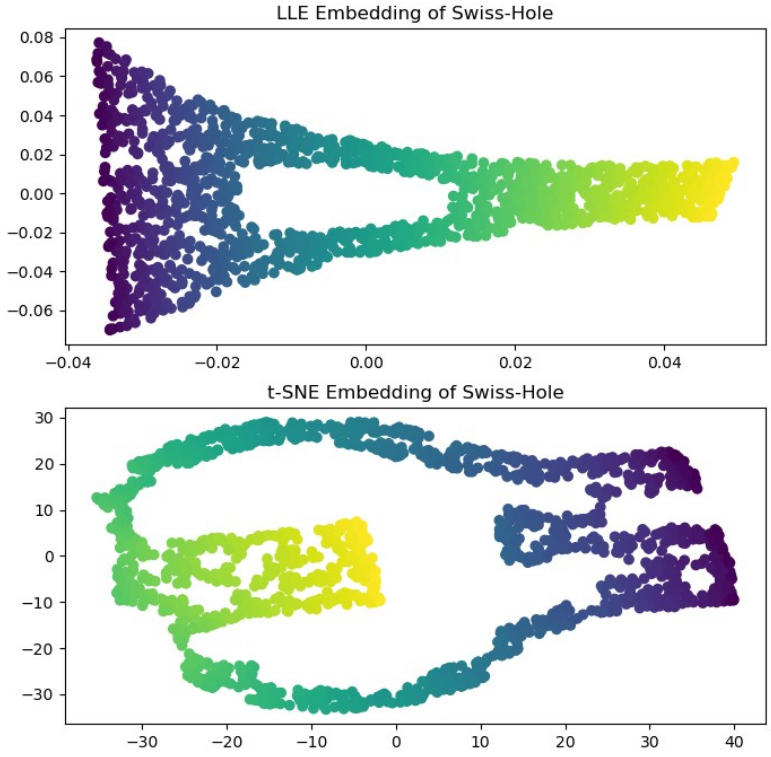

Figure 6. A lower-dimensional representation of the Swiss roll. This visual supports the idea that a simpler coordinate system can reveal structure hidden in the original view. Source: scikit-learn developers (2025), scikit-learn documentation. Citation: scikit-learn developers, 2025

4.3 Clean code example: visualizing compressed representations

The following code creates toy crack-like images, flattens them into vectors, and uses singular value decomposition to make a simple two-dimensional plot. This is not a neural network yet. It is a beginner visualization that shows how images can be represented as points.

Code Example 4. Simple representation plot for unlabeled images

```
x = make_crack_images(n_images=300, image_size=28)
flat = x.reshape(len(x), -1)      # Each image becomes one long vector.
flat = flat - flat.mean(axis=0)   # Center each pixel column.

# Use singular value decomposition to get two plotting directions.
_, _, vt = np.linalg.svd(flat, full_matrices=False)
z2 = flat @ vt[:2].T

plt.figure(figsize=(5, 4))
plt.scatter(z2[:, 0], z2[:, 1], s=10)
plt.xlabel("Direction 1")
plt.ylabel("Direction 2")
plt.title("Toy image representations in 2D")
plt.show()
```

Input meaning: x contains unlabeled image arrays. Output meaning: z2 contains two numbers per image for plotting. Mechanical engineering interpretation: if real inspection images form visible groups, an engineer should ask what caused those groups. The cause might be defect type, surface material, lighting, camera distance, or preprocessing quality.

Section summary

Unsupervised learning can reveal structure even when labels are missing. It is useful for exploring raw mechanical engineering data, but the interpretation must be cautious. A cluster in a plot is a clue, not a final engineering conclusion.

5. Theory and Mathematics for Beginner Unsupervised Learning

The mathematics in this section is intentionally light. The goal is to connect simple equations to model behavior.

5.1 What a representation is

Let x represent an input example. For an image, x may contain hundreds or thousands of pixel values. A representation z is a new vector that summarizes x. A learning system computes z = f(x), where f is a learned or chosen transformation.

$$z = f(x)$$

Here x is the raw input and z is its representation. In an autoencoder, f is usually the encoder network.

A good representation keeps information that matters for the task. For example, a crack-image representation might preserve approximate crack direction and width while ignoring small random noise. A note representation might preserve words related to urgency, damage type, and location.

5.2 What compression means

Compression means describing an input with fewer numbers. If a 28 by 28 grayscale image has 784 pixel values and an encoder compresses it into 16 numbers, the compression is strong. Compression forces the model to choose what information matters most. If the bottleneck is too small, the reconstruction may become blurry or incomplete. If the bottleneck is too large, the model may simply copy details without learning a useful summary.

```
compression_ratio = original_number_of_values / latent_number_of_values
```

For a 28 by 28 image compressed to 16 latent values, the ratio is 784 / 16 = 49.

5.3 Reconstruction error

A reconstruction is the model output that tries to reproduce the input. If x is the original input and x_hat is the reconstructed input, the reconstruction error measures how different they are. A common beginner loss is mean squared error.

```
loss = mean((x - x_hat)^2)
```

A lower value means the reconstruction is closer to the original on average. Low reconstruction error does not always mean the model understands the engineering meaning of the image.

5.4 Latent space intuition

A latent space is the coordinate system of representations. Each input becomes a point in this space. Images that are visually similar may be near each other. Moving smoothly between two points in latent space may create a smooth change in reconstructed images. This is why latent spaces are useful for interpolation, anomaly screening, and comparison.

In mechanical engineering, latent-space plots can help organize large image archives. For example, several photos of similar crack geometry may appear near each other. However, a latent space can also reflect lighting, camera resolution, or background texture. Always inspect examples, not just plots.

5.5 Why labels are not always required

Labels provide direct supervision, but structure can exist without labels. Similar images share visual patterns. Similar notes share words and phrases. An unsupervised model can learn these similarities by compressing, reconstructing, contrasting, or denoising. The model is not told the engineering category, but it can still learn useful regularities.

Section summary

Unsupervised learning is about learning useful representations from input data. Compression, reconstruction error, and latent spaces are the key ideas needed for autoencoders and many later generative workflows.

6. Autoencoders for Image Compression and Reconstruction

An autoencoder is a neural network trained to copy its input through a compressed internal bottleneck. It has three conceptual parts: an encoder, a bottleneck, and a decoder. The encoder maps the input to a

smaller latent vector. The bottleneck stores the compressed representation. The decoder maps the latent vector back to a reconstruction. Official TensorFlow documentation introduces autoencoders for basics, image denoising, and anomaly detection contexts (TensorFlow, 2024a).

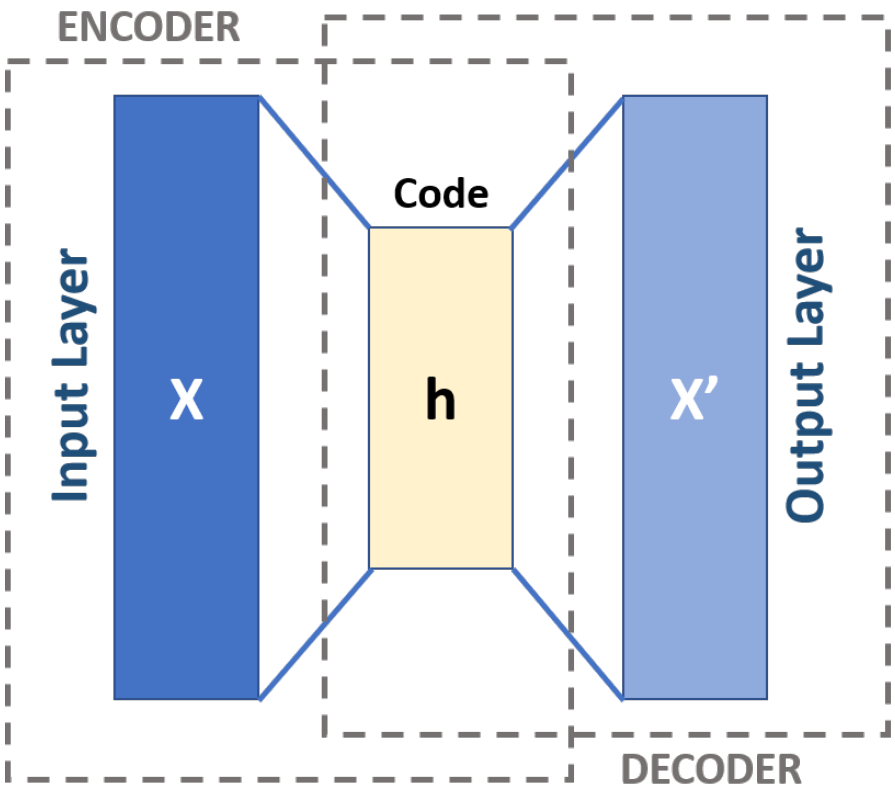

Figure 7. Autoencoder structure with encoder, compressed code, and decoder. The figure directly supports the beginner idea of input compression followed by reconstruction. Source: Michela Massi (2019), Wikimedia Commons, CC BY-SA 4.0. Citation: Massi, 2019

6.1 Purpose, inputs, and outputs

Table 7. Autoencoder parts and data flow.

Part	Input	Output	Purpose
Encoder	Original image x	Latent vector z	Compress the input into a useful internal representation.
Bottleneck	Latent vector z	Same vector z	Limit information capacity so the model must summarize.
Decoder	Latent vector z	Reconstructed image x_hat	Rebuild the input from the compressed representation.
Loss function	x and x_hat	One error value	Measure how closely the reconstruction matches the input.

Mechanical engineering interpretation: an autoencoder can be used as a first experiment for image compression, denoising, and anomaly screening. If trained mostly on normal images, examples with unusual damage may reconstruct poorly. That does not prove damage, but it can help prioritize inspection review.

6.2 Clean code example: a small image autoencoder

The following code trains a small dense autoencoder on toy crack-like images. A dense layer connects every input value to every output value. This is not the most powerful image architecture, but it is easy for beginners to understand.

Code Example 5. Training a simple autoencoder

```
# Create small unlabeled images.
x_train = make_crack_images(n_images=1200, image_size=28)
x_test = make_crack_images(n_images=120, image_size=28)

# Flatten each 28 by 28 image into 784 values.
x_train_flat = x_train.reshape(len(x_train), -1)
x_test_flat = x_test.reshape(len(x_test), -1)

latent_dim = 16

encoder = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(latent_dim, activation="relu")
], name="encoder")

decoder = keras.Sequential([
    layers.Input(shape=(latent_dim,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(784, activation="sigmoid")
], name="decoder")

inputs = keras.Input(shape=(784,))
latent = encoder(inputs)
outputs = decoder(latent)
autoencoder = keras.Model(inputs, outputs, name="toy_autoencoder")

autoencoder.compile(optimizer="adam", loss="mse")
history = autoencoder.fit(
    x_train_flat, x_train_flat,
    validation_data=(x_test_flat, x_test_flat),
    epochs=15,
    batch_size=64,
    verbose=1
)
```

The input and target are both x_train_flat. This is the defining autoencoder idea: the model learns to reconstruct the same data it receives. The bottleneck prevents perfect copying and encourages a compressed representation.

6.3 Visual comparison of original and reconstructed images

Code Example 6. Inspecting autoencoder reconstructions

```
# Reconstruct test images.
recon_flat = autoencoder.predict(x_test_flat[:10])
recon = recon_flat.reshape(-1, 28, 28, 1)

fig, axes = plt.subplots(2, 10, figsize=(10, 2.2))
for i in range(10):
    axes[0, i].imshow(x_test[i].squeeze(), cmap="gray")
    axes[0, i].axis("off")
    axes[1, i].imshow(recon[i].squeeze(), cmap="gray")
    axes[1, i].axis("off")
```

```
axes[0, 0].set_ylabel("Original")
axes[1, 0].set_ylabel("Rebuilt")
plt.suptitle("Original images versus autoencoder reconstructions")
plt.show()
```

Output meaning: the first row is the original image and the second row is the reconstruction. Mechanical engineering interpretation: if the reconstruction loses a small but important crack branch, the numerical loss may not tell the full story. Visual inspection remains important, especially for safety-related interpretation.

6.4 Latent-space sampling and interpolation

Latent-space interpolation means moving between two compressed representations and decoding the intermediate points. This helps students see that the latent space behaves like a learned map of image variation.

Code Example 7. Interpolating between two latent vectors

```
# Choose two test images and encode them.
z_a = encoder.predict(x_test_flat[0:1])
z_b = encoder.predict(x_test_flat[1:2])

# Interpolate between their latent vectors.
steps = np.linspace(0.0, 1.0, 8)
z_interp = np.array([(1 - t) * z_a[0] + t * z_b[0] for t in steps])

# Decode each intermediate latent vector.
interp_flat = decoder.predict(z_interp)
interp_imgs = interp_flat.reshape(-1, 28, 28)

plt.figure(figsize=(8, 1.4))
for i, img in enumerate(interp_imgs):
    plt.subplot(1, 8, i + 1)
    plt.imshow(img, cmap="gray")
    plt.axis("off")
plt.suptitle("Latent-space interpolation")
plt.show()
```

Mechanical engineering interpretation: interpolation is not a certified way to invent real defects. It is a teaching tool that shows the model has learned a smooth internal space. In later research, latent spaces can support similarity search and representation-based anomaly review.

6.5 Visualizing compressed representations from the encoder

Code Example 8. Plotting autoencoder latent vectors

```
# Encode all test images to latent vectors.
z = encoder.predict(x_test_flat)
z_centered = z - z.mean(axis=0)

# Project latent vectors into two dimensions for plotting.
_, _, vt = np.linalg.svd(z_centered, full_matrices=False)
z2 = z_centered @ vt[:2].T

plt.figure(figsize=(5, 4))
plt.scatter(z2[:, 0], z2[:, 1], s=20)
plt.xlabel("Latent direction 1")
plt.ylabel("Latent direction 2")
plt.title("Encoder latent representations")
plt.show()
```

Input meaning: z contains one 16-number representation for each image. Output meaning: the scatter plot shows a two-dimensional view of the latent space. Mechanical engineering interpretation: nearby points may represent similar image patterns, but an engineer should inspect original images near each point before assigning meaning.

6.6 Autoencoder checklist for beginners

Table 8. Practical checklist for classroom autoencoder experiments.

Question	What to inspect	Common beginner issue
Is the bottleneck too small?	Reconstructions are overly blurry or missing key features.	The latent vector cannot hold enough information.
Is the bottleneck too large?	Reconstructions look perfect but latent space may be less useful.	The model may copy rather than summarize.
Is the training loss decreasing?	Loss curve over epochs.	Learning rate, data scaling, or model size may be wrong.
Are reconstructions visually reasonable?	Original-reconstruction image pairs.	Numerical loss may hide important visual differences.
Does latent space show structure?	Two-dimensional projection of z.	The plot may reflect lighting or noise rather than engineering meaning.

Section summary

An autoencoder learns to compress and reconstruct. The encoder creates a latent representation, the bottleneck limits information, and the decoder reconstructs the input. For mechanical engineering, autoencoders are valuable first experiments for image compression, reconstruction, denoising, and anomaly exploration.

7. Introduction to Generative Adversarial Networks

A generative adversarial network, or GAN, is a generative model with two neural networks trained together. The generator creates fake samples from random noise. The discriminator tries to distinguish real samples from fake samples. The generator improves by trying to fool the discriminator. GANs were introduced by Goodfellow and coauthors in 2014 (Goodfellow et al., 2014). Official TensorFlow documentation provides a deep convolutional GAN tutorial using Keras and TensorFlow training loops (TensorFlow, 2024b).

7.1 Generator and discriminator basics

Table 9. Roles in a GAN.

Network	Input	Output	Beginner purpose	Mechanical Engineering analogy
Generator	Random vector z	Fake image-like sample	Learn to create samples that resemble the training images.	Create toy crack-like patterns from random numbers.
Discriminator	Real or fake image	Score or probability of being real	Learn to separate real training examples from generator outputs.	Act like a reviewer that asks whether an image looks like the toy training set.
Training loop	Batches of real images and random vectors	Updated generator and discriminator weights	Improve both networks through competition.	Classroom experiment in synthetic defect-like image generation.

7.2 Adversarial training intuition

The generator and discriminator have opposite goals. At the beginning, the generator creates poor samples. The discriminator can easily reject them. As training continues, the generator learns patterns that make its outputs more realistic. The discriminator must also improve. This competition can be powerful, but it can also be unstable.

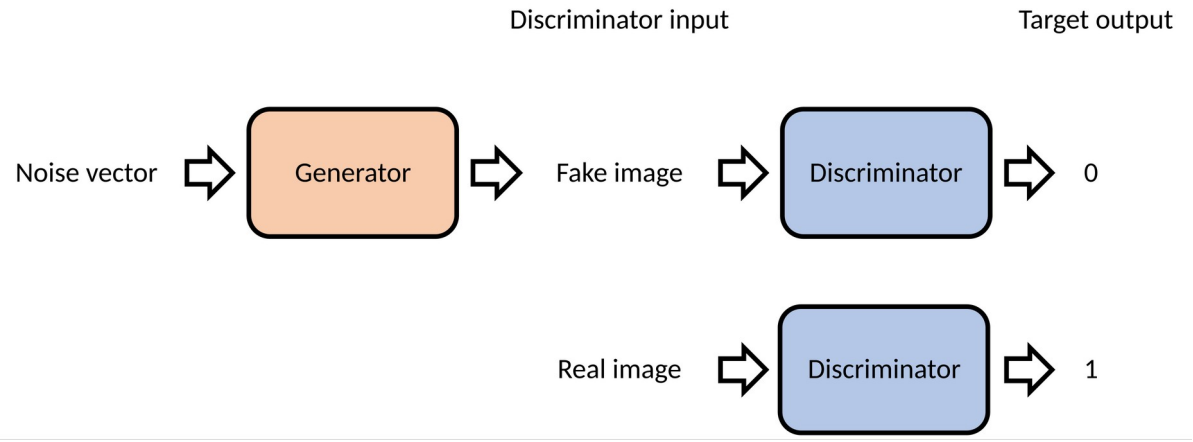

Figure 8. A second complete GAN illustration emphasizing the generator-discriminator contest. This visual is useful before code because students can identify where random input, generated output, and discriminator feedback fit. Source: Wikimedia Commons contributors (2023), CC licensed image. Citation: Wikimedia Commons, 2023

7.3 Clean code example: a lightweight GAN on toy images

The next example builds a tiny GAN for 28 by 28 toy crack-like image arrays. The model is intentionally small. It is meant to teach the workflow, not produce realistic engineering imagery.

Code Example 9. GAN data and model definitions

```
# Training data: toy crack-like images scaled to [-1, 1].
real_images = make_crack_images(n_images=1500, image_size=28)
real_images = real_images * 2.0 - 1.0
batch_size = 64
latent_dim = 32

dataset = tf.data.Dataset.from_tensor_slices(real_images)
dataset = dataset.shuffle(1500).batch(batch_size).prefetch(tf.data.AUTOTUNE)

def build_generator():
    return keras.Sequential([
        layers.Input(shape=(latent_dim,)),
        layers.Dense(7 * 7 * 64, use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Reshape((7, 7, 64)),
        layers.Conv2DTranspose(32, 3, strides=2, padding="same", use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Conv2DTranspose(16, 3, strides=2, padding="same", use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),
        layers.Conv2D(1, 3, padding="same", activation="tanh")
    ], name="generator")

def build_discriminator():
    return keras.Sequential([
```

```
        layers.Input(shape=(28, 28, 1)),
        layers.Conv2D(32, 3, strides=2, padding="same"),
        layers.LeakyReLU(),
        layers.Conv2D(64, 3, strides=2, padding="same"),
        layers.LeakyReLU(),
        layers.Flatten(),
        layers.Dense(1)
    ], name="discriminator")

generator = build_generator()
discriminator = build_discriminator()

bce = keras.losses.BinaryCrossentropy(from_logits=True)
g_opt = keras.optimizers.Adam(learning_rate=1e-4)
d_opt = keras.optimizers.Adam(learning_rate=1e-4)
```

7.4 Generator and discriminator losses

The discriminator sees both real images and fake images. It should give high scores to real images and low scores to fake images. The generator wants the discriminator to give high scores to fake images. The losses below implement that competition.

Code Example 10. One GAN training step

```
def discriminator_loss(real_logits, fake_logits):
    real_loss = bce(tf.ones_like(real_logits), real_logits)
    fake_loss = bce(tf.zeros_like(fake_logits), fake_logits)
    return real_loss + fake_loss

def generator_loss(fake_logits):
    return bce(tf.ones_like(fake_logits), fake_logits)

def train_step(real_batch):
    noise = tf.random.normal([tf.shape(real_batch)[0], latent_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        fake_batch = generator(noise, training=True)

        real_logits = discriminator(real_batch, training=True)
        fake_logits = discriminator(fake_batch, training=True)

        g_loss = generator_loss(fake_logits)
        d_loss = discriminator_loss(real_logits, fake_logits)

    g_grads = gen_tape.gradient(g_loss, generator.trainable_variables)
    d_grads = disc_tape.gradient(d_loss, discriminator.trainable_variables)

    g_opt.apply_gradients(zip(g_grads, generator.trainable_variables))
    d_opt.apply_gradients(zip(d_grads, discriminator.trainable_variables))
    return g_loss, d_loss
```

7.5 Monitoring GAN outputs over time

Code Example 11. GAN monitoring with images and losses

```
fixed_noise = tf.random.normal([16, latent_dim])
g_losses, d_losses = [], []

for epoch in range(8):
    for real_batch in dataset:
        g_loss, d_loss = train_step(real_batch)
```

```
    g_losses.append(float(g_loss))
    d_losses.append(float(d_loss))

    fake = generator(fixed_noise, training=False)
    fake = (fake + 1.0) / 2.0

    fig, axes = plt.subplots(2, 8, figsize=(8, 2))
    for ax, img in zip(axes.ravel(), fake.numpy()):
        ax.imshow(img.squeeze(), cmap="gray")
        ax.axis("off")
    plt.suptitle(f"Generated samples after epoch {epoch + 1}")
    plt.show()

plt.figure(figsize=(5, 3))
plt.plot(g_losses, label="generator")
plt.plot(d_losses, label="discriminator")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.title("GAN losses")
plt.show()
```

Output meaning: each grid displays generated samples from the same fixed noise vectors, so students can see how outputs change over training. The loss plot shows generator and discriminator losses, but visual inspection is essential. GAN losses are often harder to interpret than standard supervised losses.

7.6 Stability issues in beginner terms

Table 10. Common GAN training symptoms and beginner explanations.

Symptom	Plain-language explanation	What to try in a classroom setting
Poor samples	The generator has not learned the training pattern yet.	Train longer, use a slightly larger model, or check data scaling.
Discriminator too strong	The discriminator rejects fake samples too easily, giving weak learning signals to the generator.	Reduce discriminator capacity or train generator more often.
Generator too strong	The discriminator cannot provide useful feedback.	Check learning rates and loss curves.
Mode collapse	The generator produces many very similar samples instead of diverse outputs.	Use fixed noise previews and look for repeated outputs.
Losses look confusing	GAN losses are a game, not a simple one-direction error.	Inspect generated images at each epoch and compare with real examples.

7.7 Mechanical Engineering interpretation

A classroom GAN can create toy defect-like samples, but it should not be mistaken for a validated data-generation system. In future mechanical engineering research, generative models may help with data augmentation, scenario exploration, image restoration, or simulation support. However, synthetic mechanical systems imagery must be evaluated carefully because false realism can be misleading. A generated crack image is not field evidence.

Section summary

A GAN trains a generator and discriminator in competition. It can produce new samples but is sensitive to training balance. For beginners, the most important checks are image previews, real-versus-fake intuition, and awareness of instability.

8. Theory and Mathematics for Beginner GAN Workflows

8.1 Generator versus discriminator roles

The generator is a function G that maps a random vector z into a generated sample. The discriminator is a function D that maps an image into a score. The score is interpreted as how real the image appears to the discriminator.

```
fake_image = G(z)
```

The generator turns random input into an image-like output.

```
realness_score = D(image)
```

The discriminator assigns a score to either a real training image or a generated image.

8.2 Real versus fake predictions

During discriminator training, real images are paired with target value 1 and fake images are paired with target value 0. During generator training, fake images are paired with target value 1 because the generator wants the discriminator to believe they are real. This reversal is what makes GAN training adversarial.

Table 11. Target values used in a basic GAN.

Training part	Input to discriminator	Target used for loss	Meaning
Discriminator	Real image	1	Real images should be recognized as real.
Discriminator	Generated image	0	Generated images should be recognized as fake.
Generator	Generated image	1	The generator wants its images to be accepted as real.

8.3 Adversarial objective intuition

The original GAN objective can be written compactly, but beginners should focus on the idea. The discriminator tries to maximize correct real-versus-fake decisions. The generator tries to reduce the discriminator ability to reject generated samples. At equilibrium, the generated distribution is close to the real data distribution, in theory.

```
D tries to separate real x from fake G(z); G tries to make G(z) look real.
```

This sentence is more important for beginners than memorizing the full minimax equation.

8.4 Why GAN training can be unstable

GAN training is unstable because two networks are changing at the same time. When the discriminator improves, the generator task changes. When the generator improves, the discriminator task changes. This

moving target can produce oscillations, repeated samples, or sudden collapse. Small changes in architecture, learning rate, data scaling, or batch size can matter.

Mechanical engineering students should treat GAN outputs as experimental. The first question is not "Does the image look impressive?" The first question is "What did the model learn, what did it miss, and what validation evidence exists?"

Section summary

GAN mathematics is a game between two learned functions. The generator maps random vectors to samples, while the discriminator scores realness. The competition can create sharp samples but requires careful monitoring and cautious interpretation.

9. Introduction to Diffusion-Style Denoising

Diffusion models are a powerful family of generative models based on gradually adding noise to data and learning to reverse that process. Modern diffusion models are advanced, but the beginner idea is simple: learn to remove noise. Denoising Diffusion Probabilistic Models, or DDPMs, are one influential approach (Ho et al., 2020). Lilian Weng educational overview visualizes diffusion alongside other generative model families and explains the forward and reverse process (Weng, 2021).

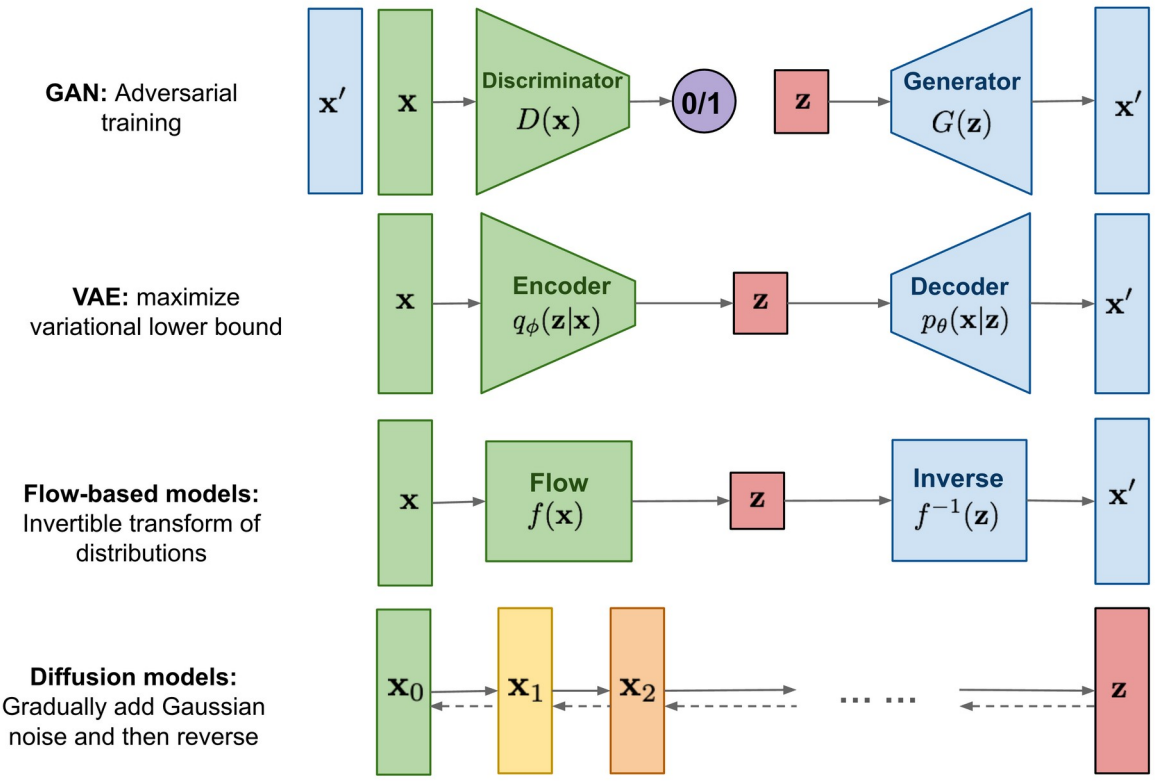

Figure 9. Generative model families, including variational autoencoders, GANs, and diffusion models. This overview helps place diffusion-style ideas in the larger generative AI landscape. Source: Weng (2021), educational technical blog. Citation: Weng, 2021

9.1 Forward noising process

The forward process gradually corrupts clean data by adding random noise. After enough steps, the image becomes mostly noise. In a full diffusion model, the model learns how to reverse this process step by step. In this beginner tutorial, we use a simpler denoising network as a stepping stone.

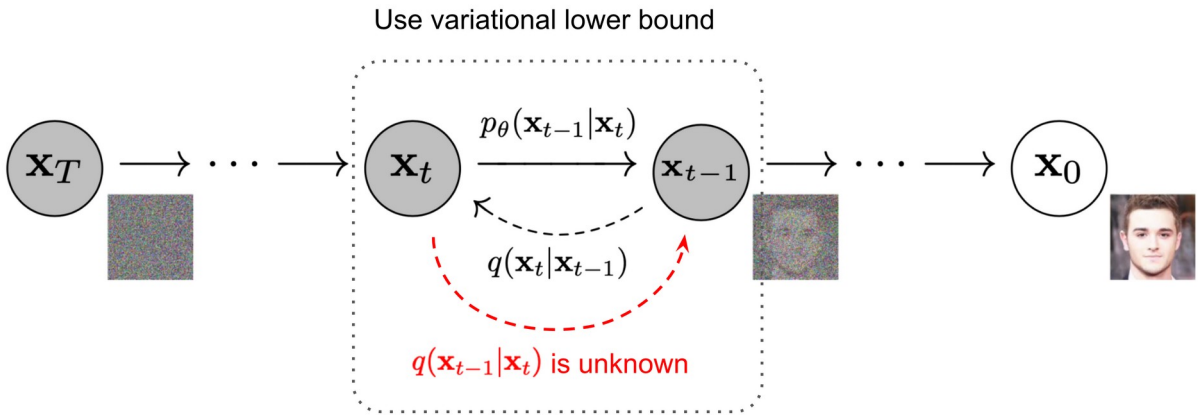

Figure 10. Forward noising and reverse denoising in a diffusion model. The complete figure shows the central idea of adding noise and then learning to remove it. Source: Weng (2021), based on Ho et al. (2020). Citation: Ho et al., 2020; Weng, 2021

9.2 Purpose, inputs, and outputs

Table 12. Diffusion-style denoising workflow for beginners.

Step	Input	Output	Purpose
Add noise	Clean image x	Noisy image x_noisy	Create a training input whose clean target is known.
Denoiser	Noisy image x_noisy	Predicted clean image or predicted noise	Learn how to reverse corruption.
Repeated denoising	Very noisy image	Sequence of cleaner images	Illustrate gradual refinement.
Visual inspection	Denoising snapshots	Qualitative understanding	Check whether important visual features are recovered.

9.3 Clean code example: adding noise and training a denoiser

This code trains a small convolutional neural network to map noisy toy images back to clean toy images. It is diffusion-style because it focuses on forward noise and learned denoising, but it is not a full production diffusion model.

Code Example 12. Training a simple denoising network

```
# Clean training images.
clean = make_crack_images(n_images=1500, image_size=28)
clean_test = make_crack_images(n_images=120, image_size=28)

def add_random_noise(images, min_sigma=0.05, max_sigma=0.65):
    sigmas = np.random.uniform(min_sigma, max_sigma, size=(len(images), 1, 1, 1))
    noisy = images + sigmas * np.random.randn(*images.shape)
    return np.clip(noisy, 0.0, 1.0).astype("float32")

noisy = add_random_noise(clean)
noisy_test = add_random_noise(clean_test)
```

```
denoiser = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, 3, padding="same", activation="relu"),
    layers.Conv2D(32, 3, padding="same", activation="relu"),
    layers.Conv2D(16, 3, padding="same", activation="relu"),
    layers.Conv2D(1, 3, padding="same", activation="sigmoid")
], name="simple_denoiser")

denoiser.compile(optimizer="adam", loss="mse")
denoiser.fit(noisy, clean, validation_data=(noisy_test, clean_test),
             epochs=8, batch_size=64)
```

9.4 Visual snapshots across noise and denoising steps

Code Example 13. Toy repeated denoising

```
# Compare clean, noisy, and denoised results.
idx = 0
start = clean_test[idx:idx + 1]
very_noisy = np.clip(start + 0.75 * np.random.randn(*start.shape), 0, 1)

snapshots = [very_noisy[0]]
current = very_noisy.astype("float32")
for step in range(5):
    current = denoiser.predict(current, verbose=0)
    snapshots.append(current[0])

plt.figure(figsize=(9, 1.6))
for i, img in enumerate(snapshots):
    plt.subplot(1, len(snapshots), i + 1)
    plt.imshow(img.squeeze(), cmap="gray")
    plt.title("start" if i == 0 else f"step {i}")
    plt.axis("off")
plt.suptitle("Repeated denoising snapshots")
plt.show()
```

Output meaning: the first image is heavily corrupted, and later images are denoised outputs. Mechanical engineering interpretation: a denoising model can help students think about image enhancement, but any use in inspection must preserve real evidence and avoid inventing damage or hiding damage.

9.5 Why this is an introductory stepping stone

A full diffusion model usually learns many time steps, often predicts noise rather than directly predicting the clean image, and samples by gradually moving from random noise to a final image. The small example above does not implement the full algorithm. It teaches the core intuition: add noise, learn to remove noise, and inspect intermediate snapshots. This is enough for beginners to understand why denoising is central to diffusion-style generative models.

Section summary

Diffusion-style workflows are based on forward noising and learned reverse denoising. The beginner code uses a simple denoiser as a stepping stone. The important ideas are noise level, denoising quality, repeated refinement, and careful visual inspection.

10. Theory and Mathematics for Beginner Diffusion-Style Workflows

10.1 Adding noise gradually

A noising process creates a corrupted version of a clean input. A simple conceptual form is shown below.

```
x_noisy = x_clean + sigma * epsilon
```

Here epsilon is random noise and sigma controls how strong the noise is. Larger sigma means more corruption.

In full DDPM-style notation, the noisy image at step t combines the original image with Gaussian noise using coefficients that depend on t. Beginners do not need to memorize the symbols at this stage. The key idea is that later steps contain more noise.

10.2 Learning to remove noise

A denoising network learns a function that maps a noisy input back toward a clean target or predicts the noise that was added. The loss compares the prediction with the known clean target or the known noise.

```
denoised = model(x_noisy)
```

The model takes a corrupted input and returns a cleaner output.

```
loss = mean((x_clean - denoised)^2)
```

This simple loss trains the denoiser in the classroom example.

10.3 Repeated denoising intuition

Repeated denoising means applying a learned model several times, or applying a sequence of models or time-conditioned steps. Each step should move the sample closer to a clean-looking data example. Modern diffusion models use carefully designed schedules and learned reverse transitions. The classroom example gives only the intuitive skeleton.

10.4 Why diffusion-style ideas are powerful

Diffusion-style methods are powerful because they turn hard generation into many smaller denoising decisions. Instead of creating a perfect image in one step, the model gradually refines a noisy sample. This supports high-quality generation in modern systems. For mechanical engineering education, the same idea motivates noise-aware image processing, uncertainty discussion, and careful distinction between enhancement and evidence.

Section summary

Diffusion-style mathematics begins with adding noise and learning to reverse it. A beginner should understand sigma, noisy inputs, denoising loss, and repeated refinement before studying full diffusion model derivations.

11. Part II: Introductory NLP, Attention, Transformers, and Multi-Modal Models

Part II moves from images to text and then to models that combine data types. Natural language processing, or NLP, is the field of using computers to work with text. Attention helps a model decide which tokens or image regions are important. Transformers use attention as a core operation. Multi-modal models combine inputs such as images and text.

Table 13. Part II learning path.

Theme	Core idea	Code activity	Mechanical Engineering relevance
NLP	Convert text into tokens, IDs, and embeddings.	Classify short inspection notes and predict next tokens.	Use maintenance logs and inspection comments.
Attention and transformers	Let each token focus on relevant tokens.	Build attention and a small transformer encoder.	Identify important words such as damage type, location, and urgency.
Multi-modal learning	Combine image and text representations.	Train a simple image-text matching model.	Link inspection photographs with textual descriptions.

12. Introduction to Natural Language Processing

Natural language processing is the use of algorithms and models to work with human language. Mechanical engineering text data can include inspection comments, condition assessment notes, work orders, laboratory observations, and maintenance logs. Text is useful because it often contains context that an image alone cannot show, such as location, date, severity, repair history, or inspector concern.

12.1 Text preprocessing and tokenization

Tokenization means splitting text into pieces called tokens. A token may be a word, part of a word, punctuation mark, or special symbol depending on the tokenizer. In a beginner TensorFlow example, a token is often a lowercase word after punctuation has been removed.

Table 14. Example text preprocessing steps.

Step	Plain-language meaning	Example
Lowercasing	Make capitalization consistent.	"Crack" becomes "crack".
Punctuation removal	Remove commas, periods, and similar marks for simple examples.	"joint," becomes "joint".
Tokenization	Split text into tokens.	"crack near joint" becomes ["crack", "near", "joint"].
Vocabulary construction	Create a list of known tokens.	crack, joint, leak, bearing, spall.
Integer encoding	Replace each token with its vocabulary index.	["crack", "joint"] becomes [5, 12].

12.2 Vocabulary construction and integer encoding

Keras provides a TextVectorization layer that can learn a vocabulary from text and convert strings to integer sequences. The official TensorFlow basic text classification tutorial demonstrates this style of workflow for beginner text classification (TensorFlow, 2024c).

Code Example 14. Tokenization and integer encoding

```
notes = np.array([
    "hairline crack at mechanical housing joint",
    "wide crack near column base",
    "water ponding on housing after thermal cycling",
    "minor surface discoloration",
    "spall observed near rebar",
    "sealant in good condition",
    "bearing area requires urgent review",
    "no visible defect on machined surface"
])

vectorizer = layers.TextVectorization(
    max_tokens=1000,
    output_sequence_length=8,
    standardize="lower_and_strip_punctuation"
)
vectorizer.adapt(notes)

encoded = vectorizer(notes[:3])
print("Vocabulary:", vectorizer.get_vocabulary()[:15])
print("Encoded notes:\n", encoded.numpy())
```

Input meaning: notes is an array of short engineering comments. Output meaning: encoded is a tensor of integer token IDs. Mechanical engineering interpretation: the workflow converts human-readable inspection notes into numbers that a neural network can process.

12.3 Embeddings

An embedding is a learned vector for a token. Instead of representing a token only by an integer ID, the model learns a vector of numbers. Tokens used in similar contexts may have embeddings that become similar during training. TensorFlow word embeddings guide presents embeddings as dense vector representations learned by a model (TensorFlow Text, 2024).

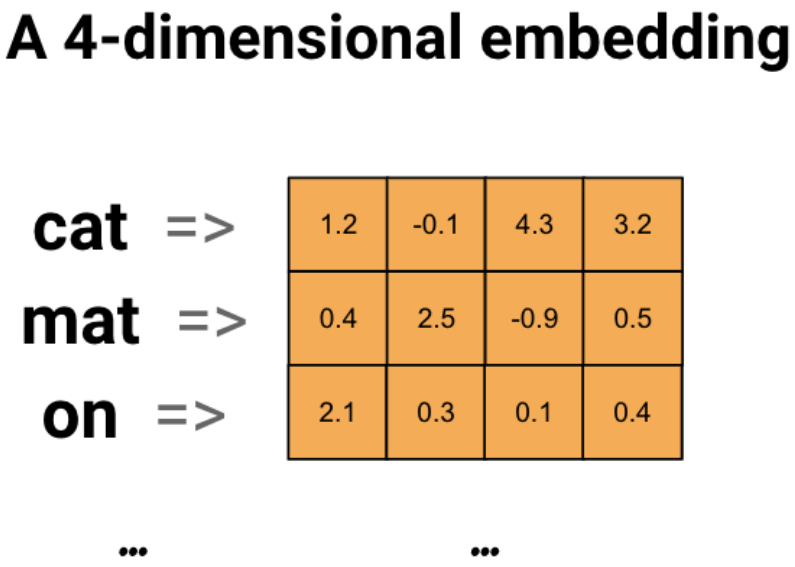

Figure 11. TensorFlow educational visualization of word embeddings. The figure supports the beginner idea that words can be represented as learned vectors rather than one-hot indicators. Source: TensorFlow Text Guide (2024), TensorFlow documentation. Citation: TensorFlow Text, 2024

12.4 Simple text classification

Text classification predicts a class from text. In a mechanical engineering classroom example, a note can be classified as requiring action or not requiring immediate action. The dataset below is tiny and only for demonstration.

Code Example 15. Beginner text classification

```
texts = np.array([
    "hairline crack at mechanical housing joint",
    "wide crack near column base",
    "water ponding on housing after thermal cycling",
    "minor surface discoloration",
    "spall observed near rebar",
    "sealant in good condition",
    "bearing area requires urgent review",
    "no visible defect on machined surface",
    "deflection observed near pressure vessel",
    "routine inspection completed",
    "corrosion staining near connection",
    "coolant return clear and functioning"
])

# 1 means action or review likely needed. 0 means no immediate action in this toy set.
labels = np.array([1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0], dtype="float32")

text_vectorizer = layers.TextVectorization(
    max_tokens=1000,
    output_sequence_length=10,
    standardize="lower_and_strip_punctuation"
)
text_vectorizer.adapt(texts)

model = keras.Sequential([
    layers.Input(shape=(1,), dtype=tf.string),
    text_vectorizer,
    layers.Embedding(input_dim=1000, output_dim=16, mask_zero=True),
    layers.GlobalAveragePooling1D(),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(texts, labels, epochs=40, verbose=0)

new_notes = np.array(["crack and spall near joint", "surface looks clean"])
print(model.predict(new_notes))
```

Output meaning: each prediction is a probability between 0 and 1. A higher value means the model thinks the note is more likely to need review in this toy setup. Mechanical engineering interpretation: real maintenance classification would require many validated examples, clear labeling policy, and domain review.

12.5 Next-token prediction at an introductory level

Next-token prediction asks a model to predict the next token in a sequence. Large language models use advanced versions of this idea, but beginners can learn the basic mechanics with a small recurrent neural network. The model below is not designed to write reports. It only demonstrates how sequences can be learned.

Code Example 16. Toy next-token prediction

```
sequence_texts = np.array([
    "crack observed at mechanical housing joint after thermal cycling",
    "spall observed near exposed rebar",
    "water ponding observed on turbine blade surface",
    "bearing requires urgent review by engineer",
    "minor discoloration observed on material surface",
    "deflection observed near pressure vessel"
])

seq_vectorizer = layers.TextVectorization(
    max_tokens=500,
    output_sequence_length=8,
    standardize="lower_and_strip_punctuation"
)
seq_vectorizer.adapt(sequence_texts)
ids = seq_vectorizer(sequence_texts).numpy()

x_seq = ids[:, :-1]  # all tokens except the last position
y_seq = ids[:, 1:]   # next token at each position
vocab_size = len(seq_vectorizer.get_vocabulary())

next_token_model = keras.Sequential([
    layers.Input(shape=(7,)),
    layers.Embedding(vocab_size, 16, mask_zero=True),
    layers.GRU(32, return_sequences=True),
    layers.Dense(vocab_size)
])

next_token_model.compile(
    optimizer="adam",
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True)
)
next_token_model.fit(x_seq, y_seq, epochs=80, verbose=0)

prompt = np.array(["crack observed at"])
prompt_ids = seq_vectorizer(prompt).numpy()[:, :-1]
logits = next_token_model.predict(prompt_ids, verbose=0)
next_id = int(np.argmax(logits[0, 2]))
print("Predicted next token:", seq_vectorizer.get_vocabulary()[next_id])
```

Input meaning: x_seq contains partial sequences and y_seq contains the next-token targets. Output meaning: the model predicts a vocabulary distribution for each position. Mechanical engineering interpretation: future report-assistance tools may use more advanced sequence modeling, but engineers must review wording, facts, and safety implications.

Section summary

NLP starts by turning text into tokens, token IDs, and embeddings. With those numerical representations, a small TensorFlow model can classify notes or predict next tokens. Engineering text is valuable because it carries context that images and sensors may not include.

13. Theory and Mathematics for Beginner NLP

13.1 Tokens and vocabulary

A token is one text unit. A vocabulary is the list of tokens the model knows. If a token is not in the vocabulary, the model may use a special unknown token. Beginner models usually map each token to an integer ID.

```
text -> tokens -> integer IDs
```

Example: "crack near joint" -> ["crack", "near", "joint"] -> [5, 9, 12].

13.2 Embeddings

An embedding matrix stores a vector for each vocabulary item. If the vocabulary has V tokens and each embedding has d numbers, the embedding matrix has shape V by d. During training, these vectors are adjusted to help the model solve its task.

```
embedding_vector = embedding_matrix[token_id]
```

The token ID selects one learned row from the embedding matrix.

13.3 Sequence modeling

A sequence model processes ordered tokens. Order matters because "water near joint" and "joint near water" may not have the same meaning in a detailed technical context. Recurrent neural networks process tokens step by step. Transformers process tokens with attention, which is introduced next.

13.4 Prediction over words or classes

A classifier predicts a class such as action needed or no immediate action. A next-token model predicts a probability distribution over the vocabulary. Both are prediction problems, but their outputs are different.

Table 15. Text classification versus next-token prediction.

Task	Input	Output	Mechanical Engineering example
Text classification	Whole note or sentence	Class probability or class label	Does this note need urgent review?
Next-token prediction	Partial sequence	Probability over vocabulary tokens	What word is likely to follow "crack observed" in a maintenance note?
Text embedding	Token or sentence	Vector representation	Compare inspection notes by meaning or topic.

Section summary

The beginner NLP pipeline is text, tokens, vocabulary, integer IDs, embeddings, and model outputs. These ideas prepare students for attention and transformer models.

14. Introduction to Attention and Transformers

Attention is a mechanism that lets a model decide which parts of an input are most relevant. In text, a token can focus on other tokens. In image captioning, a word can focus on image regions. Transformers use attention as a central building block and were introduced in the paper "Attention Is All You Need" (Vaswani

et al., 2017). The TensorFlow transformer tutorial explains self-attention and includes educational attention visualizations (TensorFlow, 2024d).

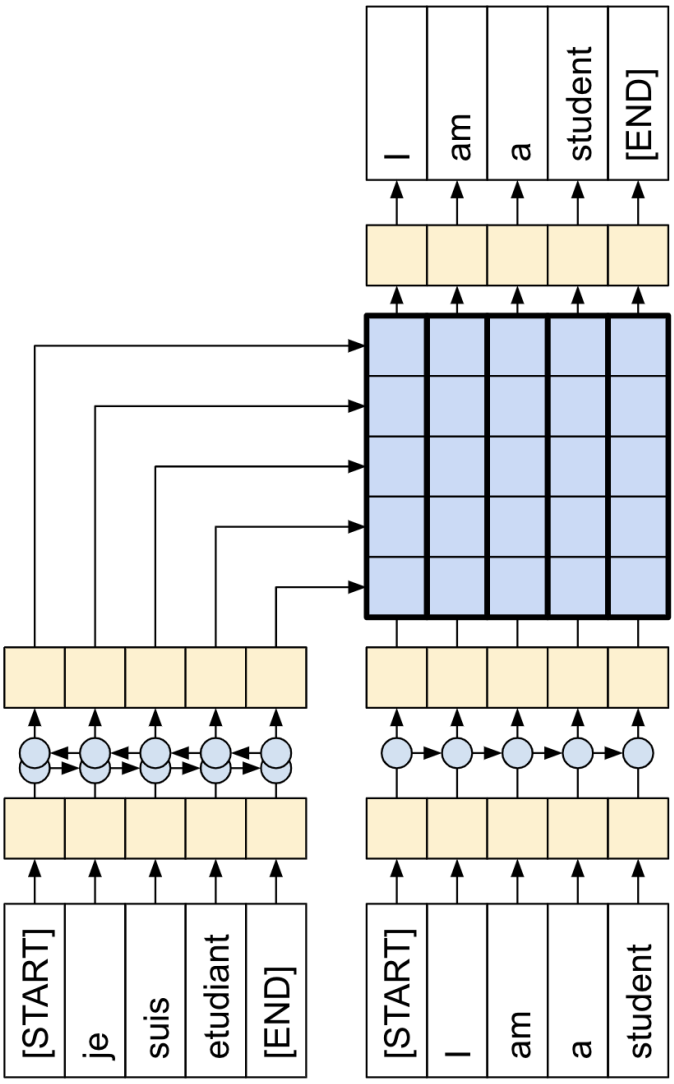

Figure 12. TensorFlow educational diagram showing recurrent processing with attention. It supports the idea that a model can use attention to focus on useful words. Source: TensorFlow Transformer tutorial (2024), TensorFlow documentation. Citation: TensorFlow, 2024d

14.1 Attention weights in plain language

Attention weights are numbers that say how much one part of the input should focus on another part. If the sentence is "wide crack near column base," the word "crack" may attend strongly to "wide" and "base" because those words help describe severity and location. Attention weights are not perfect explanations, but they are useful visual clues for beginners.

Table 16. Attention terms explained simply.

Term	Plain-language meaning	Inspection-note analogy
Query	The token asking what information it needs.	The token "crack" asks what nearby words describe it.
Key	A token label used for matching.	Each word offers a way to be matched

		by other words.
Value	The information taken from a token after matching.	The description carried from words such as "wide" or "joint".
Attention score	How well a query matches a key.	How relevant "wide" is to "crack".
Attention weight	A normalized score between tokens.	How much focus one word gives another word.
Context vector	The weighted combination of values.	A new representation of "crack" enriched by relevant surrounding words.

14.2 Self-attention

Self-attention means that tokens in the same sequence attend to one another. This allows every token to gather context from the whole sentence. For inspection notes, self-attention can connect damage words with location words, severity words, and action words.

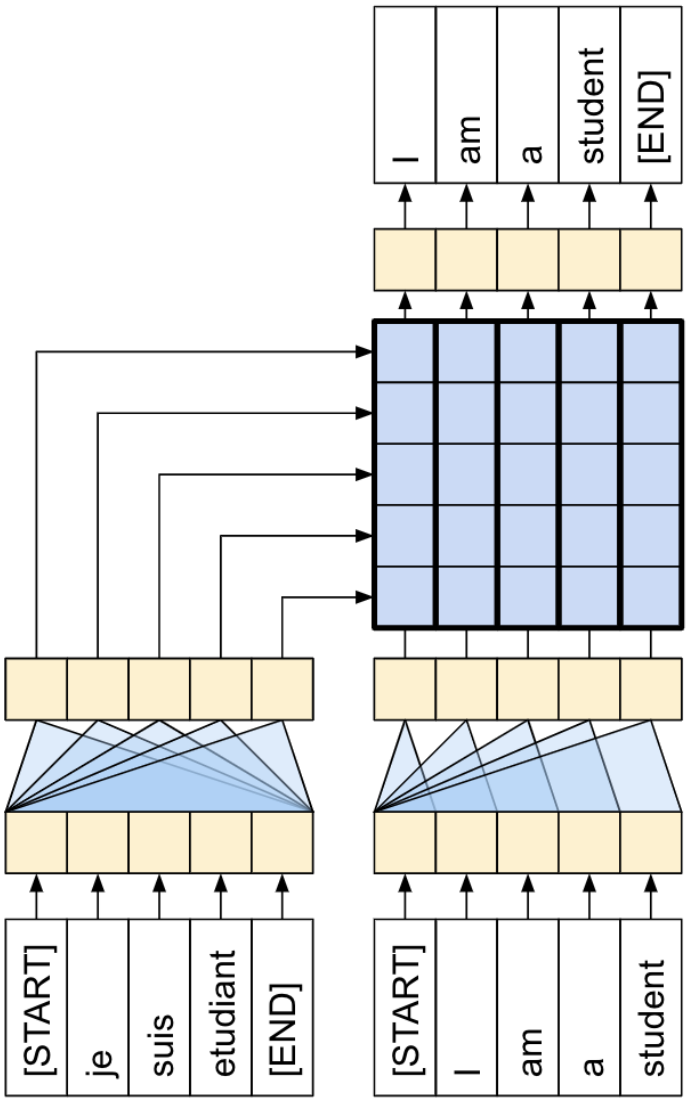

Figure 13. TensorFlow educational diagram of a one-layer transformer processing words. It illustrates self-attention inside a transformer-style block. Source: TensorFlow Transformer tutorial (2024), TensorFlow documentation. Citation: TensorFlow, 2024d

14.3 Clean code example: a small self-attention block and attention heatmap

The code below creates token embeddings and passes them through a Keras MultiHeadAttention layer. The attention weights are plotted as a heatmap. Because the layer is not trained in this small demonstration, the heatmap shows mechanics rather than a trusted engineering explanation.

Code Example 17. Visualizing self-attention weights

```
sentence = np.array(["wide crack near column base after thermal cycling"])
attn_vectorizer = layers.TextVectorization(
    max_tokens=100,
    output_sequence_length=8,
    standardize="lower_and_strip_punctuation"
)
attn_vectorizer.adapt(sentence)

token_ids = attn_vectorizer(sentence)
vocab = attn_vectorizer.get_vocabulary()
tokens = [vocab[i] for i in token_ids.numpy()[0] if i != 0]

embedding = layers.Embedding(input_dim=100, output_dim=16)
x = embedding(token_ids)

mha = layers.MultiHeadAttention(num_heads=1, key_dim=16)
context, scores = mha(x, x, return_attention_scores=True)

weights = scores.numpy()[0, 0, :len(tokens), :len(tokens)]
plt.figure(figsize=(5, 4))
plt.imshow(weights)
plt.xticks(range(len(tokens)), tokens, rotation=45)
plt.yticks(range(len(tokens)), tokens)
plt.colorbar(label="attention weight")
plt.title("Self-attention weights")
plt.tight_layout()
plt.show()
```

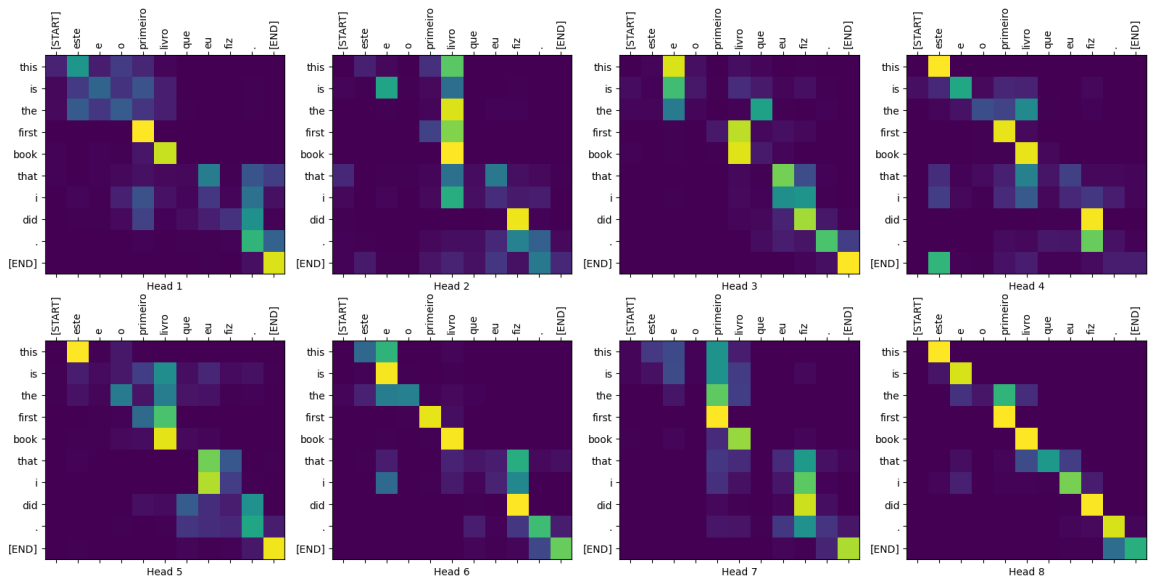

Figure 14. A complete attention map from the TensorFlow transformer tutorial. The visual shows how attention can be displayed as a heatmap between input and output tokens in a trained sequence model. Source: TensorFlow Transformer tutorial (2024), TensorFlow documentation. Citation: TensorFlow, 2024d

14.4 Introductory transformer encoder

A transformer encoder block usually contains self-attention, a feed-forward neural network, residual connections, and normalization. Positional information is needed because attention alone does not automatically know token order. A residual connection adds the input of a sublayer to its output, which helps training. A feed-forward sublayer applies a small neural network to each token representation.

Table 17. Transformer encoder pieces in beginner language.

Piece	Purpose	Beginner interpretation
Token embedding	Turn token IDs into vectors.	Each word becomes numbers.
Positional embedding	Provide information about token order.	The model knows first, second, third, and later positions.
Self-attention	Let each token focus on other tokens.	Damage words can connect with severity and location words.
Residual connection	Add the input back to a sublayer output.	Keep old information while adding new information.
Normalization	Stabilize values during training.	Make learning more stable.
Feed-forward layer	Apply a small neural network to each token.	Refine each token representation after attention.

Code Example 18. A small transformer encoder classifier

```
class TransformerEncoder(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim):
        super().__init__()
        self.attn = layers.MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
        self.ffn = keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim)
        ])
        self.norm1 = layers.LayerNormalization()
        self.norm2 = layers.LayerNormalization()

    def call(self, x):
        attn_output = self.attn(x, x)
        x = self.norm1(x + attn_output)      # residual connection
        ffn_output = self.ffn(x)
        return self.norm2(x + ffn_output)    # residual connection

max_len = 10
embed_dim = 24
num_heads = 2
ff_dim = 32
vocab_limit = 1000

text_input = keras.Input(shape=(1,), dtype=tf.string)
x = text_vectorizer(text_input)
positions = tf.range(start=0, limit=max_len, delta=1)

word_embed = layers.Embedding(vocab_limit, embed_dim, mask_zero=True)(x)
pos_embed = layers.Embedding(input_dim=max_len, output_dim=embed_dim)(positions)
x = word_embed + pos_embed
x = TransformerEncoder(embed_dim, num_heads, ff_dim)(x)
x = layers.GlobalAveragePooling1D()(x)
x = layers.Dense(16, activation="relu")(x)
output = layers.Dense(1, activation="sigmoid")(x)

transformer_classifier = keras.Model(text_input, output)
transformer_classifier.compile(
```

```
    optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"]
)
transformer_classifier.fit(texts, labels, epochs=40, verbose=0)
print(transformer_classifier.predict(new_notes))
```

Input meaning: the model receives raw text strings. Output meaning: it predicts a probability for the toy action-needed label. Mechanical engineering interpretation: transformers can model relationships between words in inspection notes, but a small classroom model needs much more data and validation before real use.

Section summary

Attention lets tokens focus on relevant tokens. Transformers combine token embeddings, positional information, self-attention, residual connections, normalization, and feed-forward layers. Attention heatmaps can support learning, but they should not be overinterpreted as complete explanations.

15. Theory and Mathematics for Beginner Attention and Transformers

15.1 Why attention is useful

Attention is useful because not every part of an input is equally relevant. In an inspection note, "urgent" may matter more than "after" for priority classification. In an image captioning system, a generated word may depend on a specific image region. Attention gives the model a learned way to focus.

15.2 How one token can focus on other tokens

Each token representation is transformed into a query, key, and value. A query is compared with keys from all tokens. The scores are normalized into attention weights. The output is a weighted combination of value vectors.

```
score_ij = dot(query_i, key_j) / sqrt(d_k)
```

This score measures how much token i should focus on token j. The denominator keeps scores numerically stable.

```
weight_ij = softmax(score_ij across j)
```

Softmax turns scores into nonnegative weights that sum to 1.

```
context_i = sum_j weight_ij * value_j
```

The context vector is a weighted mixture of information from other tokens.

15.3 What attention weights mean

An attention weight is a learned focus value. A high weight means one token drew more information from another token during that layer and head. It does not prove causation and it does not automatically produce a legally reliable explanation. For learning, heatmaps are useful because they make the invisible attention calculation visible.

15.4 Transformer-block intuition

A transformer block first lets tokens exchange information with self-attention. It then uses a feed-forward network to refine each token representation. Residual connections preserve information, while normalization helps training. Multiple blocks can be stacked, but this tutorial uses one small block to keep the workflow lightweight.

Section summary

The core attention calculation uses queries, keys, values, softmax weights, and weighted sums. A transformer block wraps attention with feed-forward refinement, residual connections, and normalization.

16. Introduction to Multi-Modal Models

A multi-modal model combines more than one type of input. A mechanical engineering example is an inspection photograph paired with a text note. The image may show a defect pattern, while the note may describe location, urgency, or historical context. Multi-modal learning can help connect these data sources. CLIP, short for Contrastive Language-Image Pre-training, is a well-known image-text model that learns from image and text pairs (Radford et al., 2021). TensorFlow image captioning tutorial also shows how image features and text tokens can be combined with attention (TensorFlow, 2024e).

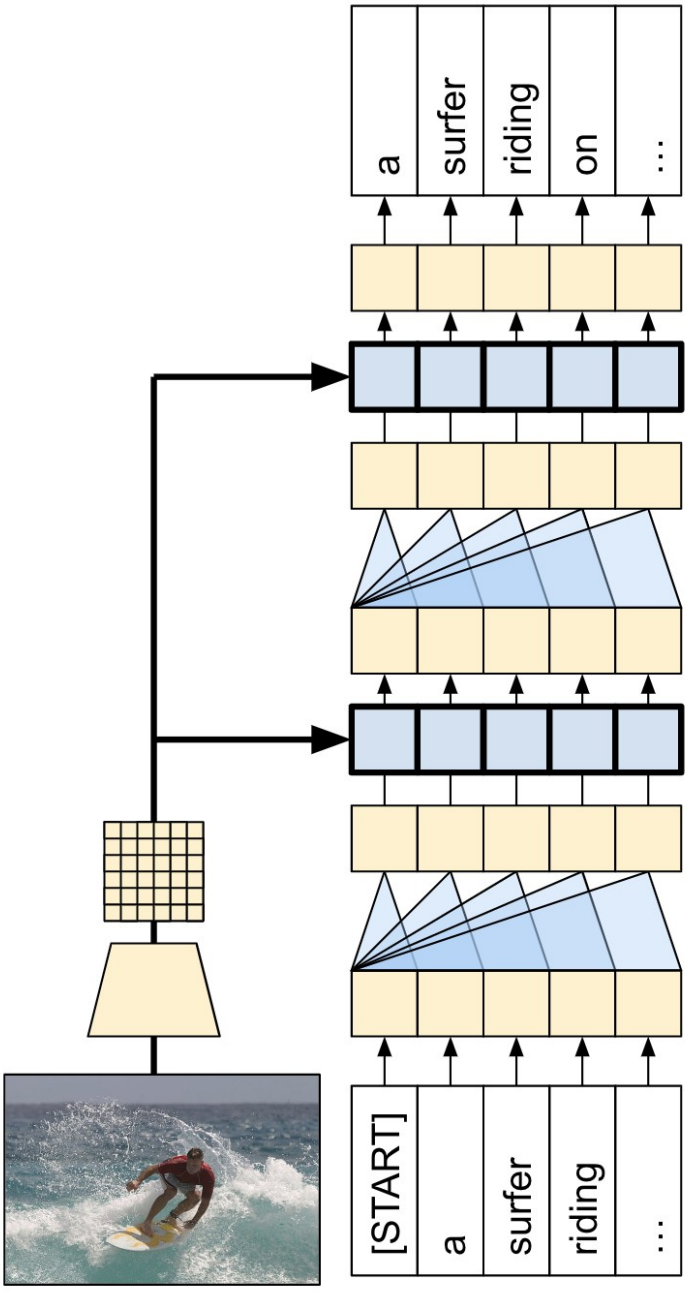

Figure 15. TensorFlow image captioning architecture. The complete diagram shows how image features and text tokens can be combined in an image-to-text workflow. Source: TensorFlow Image Captioning tutorial (2024), TensorFlow documentation. Citation: TensorFlow, 2024e

16.1 Combining image and text inputs

The simplest beginner approach is feature fusion. One branch processes the image. Another branch processes the text. The model concatenates, or joins, the two feature vectors and predicts an output. This is much simpler than large image-text systems, but it teaches the core input-output structure.

Table 18. Simple image-text fusion workflow.

Branch	Input	Processing	Output representation
Image branch	28 by 28 grayscale image	Small convolutional layers	Image feature vector

		and pooling	
Text branch	Inspection note string	TextVectorization, embedding, pooling	Text feature vector
Fusion layer	Image and text feature vectors	Concatenate and dense layers	Joint image-text representation
Prediction head	Joint representation	Sigmoid output	Matched-pair probability

16.2 Matched versus mismatched image-text pairs

A matched pair means the text describes the image. A mismatched pair means the text and image do not correspond. A simple training task is to predict 1 for matched pairs and 0 for mismatched pairs. This teaches how a model can use two data types together.

Code Example 19. Creating matched and mismatched toy pairs

```
def make_blank_images(n_images=512, image_size=28, noise=0.04):
    images = noise * np.random.randn(n_images, image_size, image_size, 1)
    return np.clip(images, 0.0, 1.0).astype("float32")

n = 600
crack_imgs = make_crack_images(n // 2, 28)
blank_imgs = make_blank_images(n // 2, 28)
images = np.concatenate([crack_imgs, blank_imgs], axis=0)

crack_text = ["crack visible near joint"] * (n // 2)
blank_text = ["surface appears normal"] * (n // 2)
texts_all = np.array(crack_text + blank_text)

# Create matched examples.
matched_images = images.copy()
matched_texts = texts_all.copy()
matched_labels = np.ones(n, dtype="float32")

# Create mismatched examples by reversing the text order.
mismatched_images = images.copy()
mismatched_texts = texts_all[::-1].copy()
mismatched_labels = np.zeros(n, dtype="float32")

pair_images = np.concatenate([matched_images, mismatched_images], axis=0)
pair_texts = np.concatenate([matched_texts, mismatched_texts], axis=0)
pair_labels = np.concatenate([matched_labels, mismatched_labels], axis=0)

# Shuffle pairs together.
idx = np.random.permutation(len(pair_labels))
pair_images, pair_texts, pair_labels = pair_images[idx], pair_texts[idx], pair_labels[idx]
```

16.3 Clean code example: simple image-text fusion model

Code Example 20. A small TensorFlow image-text fusion model

```
mm_vectorizer = layers.TextVectorization(
    max_tokens=100,
    output_sequence_length=6,
    standardize="lower_and_strip_punctuation"
)
mm_vectorizer.adapt(pair_texts)

image_input = keras.Input(shape=(28, 28, 1), name="image_input")
xi = layers.Conv2D(16, 3, activation="relu", padding="same")(image_input)
xi = layers.MaxPooling2D()(xi)
```

```
xi = layers.Conv2D(32, 3, activation="relu", padding="same")(xi)
xi = layers.GlobalAveragePooling2D()(xi)
xi = layers.Dense(24, activation="relu")(xi)

text_input = keras.Input(shape=(1,), dtype=tf.string, name="text_input")
xt = mm_vectorizer(text_input)
xt = layers.Embedding(input_dim=100, output_dim=16, mask_zero=True)(xt)
xt = layers.GlobalAveragePooling1D()(xt)
xt = layers.Dense(24, activation="relu")(xt)

combined = layers.Concatenate()([xi, xt])
combined = layers.Dense(24, activation="relu")(combined)
match_output = layers.Dense(1, activation="sigmoid")(combined)

fusion_model = keras.Model(
    inputs={"image_input": image_input, "text_input": text_input},
    outputs=match_output
)
fusion_model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

fusion_model.fit(
    {"image_input": pair_images, "text_input": pair_texts},
    pair_labels,
    epochs=8,
    batch_size=64,
    validation_split=0.2
)
```

Input meaning: the model receives an image and a text string at the same time. Output meaning: the model predicts a probability that the pair is matched. Mechanical engineering interpretation: a future asset-management workflow might compare uploaded inspection photos with notes, but real deployment would require robust datasets, metadata, privacy controls, and engineering review.

16.4 Inspecting simple multi-modal outputs

Code Example 21. Inspecting image-text match scores

```
test_images = np.concatenate([crack_imgs[:2], blank_imgs[:2]], axis=0)
test_texts = np.array([
    "crack visible near joint",    # expected match with crack image
    "surface appears normal",      # likely mismatch with crack image
    "surface appears normal",      # expected match with blank image
    "crack visible near joint"     # likely mismatch with blank image
])

scores = fusion_model.predict(
    {"image_input": test_images, "text_input": test_texts}, verbose=0
).ravel()

for text, score in zip(test_texts, scores):
    print(f"{text:30s}  match score = {score:.3f}")
```

Students should inspect both the images and the text. A high match score is not evidence that an actual defect exists. It only means the small model thinks the image and phrase fit its toy training pattern.

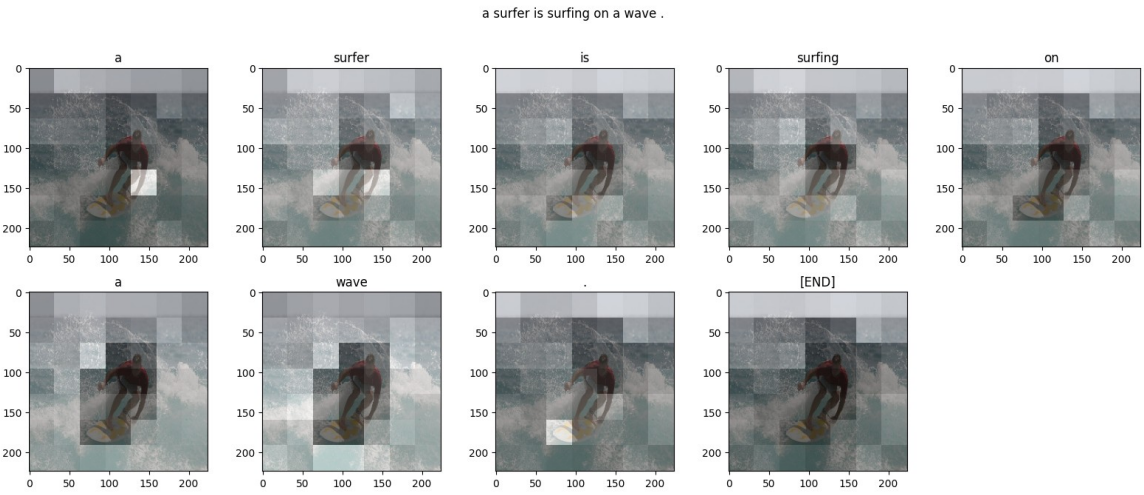

Figure 16. TensorFlow image captioning example showing visual attention during generated text prediction. This complete figure illustrates that image regions and words can interact in multi-modal workflows. Source: TensorFlow Image Captioning tutorial (2024), TensorFlow documentation. Citation: TensorFlow, 2024e

16.5 Why multi-modal learning is useful in Mechanical Engineering

- **Inspection context:** An image may show a crack, while text may identify the asset, member, location, or recent weather. Together they are more informative.
- **Quality control:** A model can flag possible mismatches, such as a note saying "no visible defect" paired with an image showing obvious distress.
- **Search and retrieval:** Image-text representations can support searching an archive for "housing cracking near drain" or similar descriptions.
- **Report support:** Multi-modal systems may help summarize image evidence and notes, but final reports require professional review.
- **Learning mounting base:** A small fusion model prepares students to understand larger systems such as captioning and contrastive image-text models.

Section summary

A multi-modal model combines data types. A simple TensorFlow image-text fusion model teaches branches, embeddings, concatenation, and matched-pair outputs. This is a mechanical component between beginner neural networks and future mechanical systems interpretation systems.

17. Mechanical Engineering Synthesis, Responsible Use, and Practice Projects

This final teaching section connects the model families. The main educational message is that unlabeled data can be useful. It can reveal structure, compress information, reconstruct inputs, denoise signals, generate samples, represent text, focus attention, and combine images with notes. These skills are foundational for future mechanical engineering AI workflows.

Table 19. Model family comparison for Mechanical Engineering students.

Model or workflow	Best beginner question	Input	Output	Mechanical Engineering mini-case

Representation plot	Do examples form groups?	Unlabeled images or vectors	2D visualization	Explore machined surface image archive before labeling.
Autoencoder	Can the model compress and reconstruct?	Image or signal	Latent vector and reconstruction	Compress and reconstruct crack-like images.
GAN	Can a model create new sample-like images?	Random vector and real image set	Generated image	Create toy defect-like samples for classroom discussion.
Diffusion-style denoiser	Can a model reverse noise?	Noisy image	Denoised image or snapshots	Clean noisy visual records while preserving evidence.
Text classifier	Can a note be assigned a useful category?	Text note	Class probability	Flag notes that may need action.
Next-token model	What token is likely next?	Partial text sequence	Vocabulary distribution	Understand language modeling basics.
Attention	Which tokens focus on which other tokens?	Token sequence	Attention weights and context vectors	Connect damage words to severity and location words.
Transformer encoder	Can attention-based representations improve text prediction?	Text sequence	Class probability or representation	Classify maintenance notes with context.
Image-text fusion	Do the image and note match?	Image plus text	Matched-pair probability	Flag inconsistent inspection records.

17.1 Suggested classroom practice projects

- Train the autoencoder with latent_dim values of 4, 8, 16, and 32. Compare remanufacturing quality and compression ratio.
- Modify the toy crack generator to include horizontal, vertical, and diagonal line patterns. Study whether the latent plot changes.
- Train the denoiser with different noise ranges and inspect whether repeated denoising removes real features.
- Add five more engineering notes to the text classifier and examine which words influence predictions through controlled examples.
- Replace the multi-modal text phrases with more detailed phrases such as "long crack near joint" and "clean surface with no visible distress." Study whether the match model improves.
- Write a one-page reflection explaining why generated images should not be treated as field evidence.

17.2 Responsible use principles

Table 20. Responsible use principles for beginner mechanical engineering AI work.

Principle	Meaning for this tutorial	Professional implication
Data quality	Small toy datasets teach mechanics but do not represent real mechanical systems variability.	Use validated datasets before claiming engineering performance.
Human review	Model outputs are suggestions or demonstrations.	Engineers and inspectors remain responsible for decisions.
Uncertainty	A probability or reconstruction loss is not absolute truth.	Report uncertainty and limitations clearly.

No hidden evidence changes	Denoising and generation can alter visual appearance.	Do not use altered imagery as unqualified evidence.
Bias and coverage	A model learns from the data it sees.	Check whether asset types, materials, lighting, and damage modes are sufficiently covered.
Documentation	Record data preparation, model settings, and evaluation.	Reproducibility is essential for professional credibility.

17.3 Final summary

Unsupervised learning begins with unlabeled data and learned structure. Autoencoders compress and reconstruct. GANs generate through a generator-discriminator contest. Diffusion-style workflows add noise and learn denoising. NLP turns engineering text into tokens, integer IDs, and embeddings. Attention lets tokens focus on relevant tokens. Transformers package attention into powerful sequence-processing blocks. Multi-modal models combine data types, such as images and notes. For Mechanical Engineering students, the key is not to memorize advanced formulas. The key is to understand the input, output, representation, loss, visualization, and engineering interpretation of each workflow.

18. References

Bahdanau, D., Cho, K., and Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. arXiv:1409.0473. https://arxiv.org/abs/1409.0473

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). Generative adversarial nets. Advances in Neural Information Processing Systems. https://arxiv.org/abs/1406.2661

Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. Advances in Neural Information Processing Systems. https://arxiv.org/abs/2006.11239

Keras. (2023). DCGAN to generate face images. Keras code examples. https://keras.io/examples/generative/dcgan_overriding_train_step/

Keras. (2024a). Keras code examples. https://keras.io/examples/

Keras. (2024b). Text classification with Transformer. Keras code examples. https://keras.io/examples/nlp/text_classification_with_transformer/

Massi, M. (2019). Autoencoder schema. Wikimedia Commons. CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Autoencoder_schema.png

National Vehicle systems Safety Board. (2018). Photo of cracks on Miami Mechanical component. Wikimedia Commons. Public domain. https://commons.wikimedia.org/wiki/File:Course-generated-machined-component-crack-image

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision. arXiv:2103.00020. https://arxiv.org/abs/2103.00020

scikit-learn developers. (2023a). Unsupervised learning. scikit-learn documentation. https://scikit-learn.org/1.3/unsupervised_learning.html

scikit-learn developers. (2024). Comparison between K-Means and MiniBatchKMeans. scikit-learn documentation. https://sklearn.org/stable/auto_examples/cluster/plot_mini_batch_kmeans.html

scikit-learn developers. (2025). Manifold learning examples. scikit-learn documentation. https://scikit-learn.org/stable/auto_examples/manifold/plot_swissroll.html

Course-generated illustrative machined-surface texture image used for this Mechanical Engineering course conversion.

TensorFlow. (2024a). Intro to autoencoders. TensorFlow Core tutorials. https://www.tensorflow.org/tutorials/generative/autoencoder

TensorFlow. (2024b). Deep convolutional generative adversarial network. TensorFlow Core tutorials. https://www.tensorflow.org/tutorials/generative/dcgan

TensorFlow. (2024c). Basic text classification. TensorFlow Core tutorials. https://www.tensorflow.org/tutorials/keras/text_classification

TensorFlow. (2024d). Neural machine translation with a Transformer and Keras. TensorFlow Text tutorials. https://www.tensorflow.org/text/tutorials/transformer

TensorFlow. (2024e). Image captioning with visual attention. TensorFlow Text tutorials. https://www.tensorflow.org/text/tutorials/image_captioning

TensorFlow Text. (2024). Word embeddings. TensorFlow Text guide. https://www.tensorflow.org/text/guide/word_embeddings

The Original Benny C. (2023). Artificial Intelligence relation to Generative Models subset, Venn diagram. Wikimedia Commons. CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:AI_relation_to_Generative_Models_subset,_venn_diagram.png

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, N., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems. https://arxiv.org/abs/1706.03762

Weng, L. (2021). What are diffusion models? Lilian Weng Blog. https://lilianweng.github.io/posts/2021-07-11-diffusion-models/

Wikimedia Commons contributors. (2023). Generative Adversarial Network illustration. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Generative_Adversarial_Network_illustration.svg

Xu, K., Ba, J., Kiros, R., Cho, K., Courville, A., Salakhutdinov, R., Zemel, R., and Bengio, Y. (2015). Show, attend and tell: Neural image caption generation with visual attention. Proceedings of ICML. https://arxiv.org/abs/1502.03044

Zhang, A., Lipton, Z. C., Li, M., and Smola, A. J. (2023). Dive into Deep Learning. GAN visual diagram via Wikimedia Commons. CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Generative_adversarial_network.svg